A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

To Do List

IN

Python

A mini project report submitted
In partial fulfilment of the requirements for the Degree of Bachelor of Technology

In
Information Technology
Semester – V

By

ANIKET PARMAR (12402080601003)

Faculty Guide
Asst Prof. Nayan Mali

Academic Year 2026-27
Department of Information Technology
AD Patel Institute of Technology

CERTIFICATE

This is to certify that **Mr. Parmar Aniket Ashvinkumar**, Enrollment Number **12402080601003**, a student of **B.Tech in Information Technology** at **A. D. Patel Institute of Technology**, has successfully completed the project entitled **"Python To-Do List Application"** as part of the academic requirements of the course.

The project has been developed using Python programming language and the Tkinter library for graphical user interface development. It demonstrates task management operations such as adding tasks, marking tasks as completed, deleting tasks, and storing task information using JSON file handling.

The work presented in this report is submitted for academic evaluation. To the best of our knowledge, the project work has been carried out by the student under the guidance and supervision of the concerned faculty member.

Guide	**Head Of Department**
Nayan Mali	**Dr.Priyang Bhatt**
(Assistant Prof.)	**(Associate Prof. & Head)**

Acknowledgement

I would like to express my sincere gratitude to **A. D. Patel Institute of Technology** for providing me with the opportunity to undertake the project titled **"Python To-Do List Application."**

I am thankful to my project guide and faculty members of the Information Technology Department for their valuable guidance, encouragement, and support throughout the development of this project.

I would also like to thank the institute for providing the necessary academic resources and learning environment that helped me understand Python programming, graphical user interface development using Tkinter, and file handling using JSON.

This project has helped me improve my programming skills and gain practical knowledge of developing a desktop application. Finally, I would like to thank my family and friends for their constant encouragement and support during the completion of this project.

ANIKET PARMAR
(12402080601003)

ABSTRACT

The Python To-Do List Application is a desktop task-management program developed using Python and the Tkinter GUI library. It allows users to add tasks, mark tasks as completed or pending, delete selected tasks, and clear all completed tasks. The application stores task information in a local JSON file named tasks.json so that tasks can be loaded again when the program starts. The project demonstrates fundamental programming concepts, graphical user interface design, file handling, JSON serialization, input validation, and the use of functions to organize a program into manageable parts.

TABLE OF CONTENTS

Section	Title
1	Introduction
2	Problem Statement, Aim and Objectives
3	Scope and Feasibility
4	System Requirements
5	Technologies Used
6	System Analysis and Design
7	System Workflow and Algorithms
8	Implementation and Code Explanation
9	Testing and Results
10	Limitations, Future Enhancements, Conclusion and References

1. INTRODUCTION

1.1 Background

People commonly manage activities such as assignments, appointments, study sessions, and household work. When these activities are kept only in memory or scattered notes, tasks may be forgotten or their progress may be difficult to track. A to-do list provides a central place to record tasks and indicate which activities are complete.

1.2 Project Overview

The application is a lightweight desktop program with a main window, a text-entry field, an Add button, a list of tasks, a completion counter, and action buttons. The user enters a task description and adds it to the list. A selected task can be marked done or returned to pending status. Users can also delete one selected task or remove all completed tasks.

1.3 Need for the Project

- Provide a simple digital alternative to paper-based task lists.
- Keep pending and completed tasks visible in one window.
- Preserve tasks when the program is closed and opened again.
- Demonstrate practical use of Python GUI and file-handling features.
- Offer a small application that works locally without internet access.

1.4 Report Organisation

This report describes the objectives and scope, tools and requirements, system design, implementation functions, expected test results, limitations, and possible future improvements.

2. PROBLEM STATEMENT, AIM AND OBJECTIVES

2.1 Problem Statement

A user needs a convenient way to enter tasks, track completion, and retain task information between sessions. A solution should be easy to operate, provide feedback when the user submits invalid input, and not require a separate database server or online account.

2.2 Aim

The aim is to design and implement a simple desktop To-Do List Application using Python, Tkinter, and JSON file storage.

2.3 Objectives

1. Create a graphical window for task management.
2. Accept task descriptions through an entry field.
3. Prevent empty or whitespace-only task descriptions.
4. Represent each task with its text and completion status.
5. Allow a task to be marked completed or returned to pending.
6. Allow deletion of a selected task.
7. Remove all completed tasks through one action.
8. Save task data to a local JSON file.
9. Load saved tasks when the application starts.
10. Show the completed-task count and total task count.

2.4 Success Criteria

The project meets its core goals when a user can add a valid task, see it in the list, change its status, delete it, clear completed tasks, and reopen the program with saved tasks restored. Invalid actions such as adding an empty task or toggling without a selection should produce a warning.

3. SCOPE AND FEASIBILITY

3.1 Project Scope

The current version is intended for personal task tracking on one computer. It provides basic create, update, and delete operations for task records. It stores information in tasks.json in the application's working directory and updates the interface after changes.

3.2 Included Features

- Add tasks using the Add button or Enter key.
- Display pending and completed tasks with symbols.
- Toggle a task's completion status.
- Delete a selected task.
- Clear every task marked completed.
- Show the completion count.
- Load and save tasks using JSON.
- Show warnings for empty input and missing selections.

3.3 Outside the Current Scope

- Multiple users, authentication, and access permissions.
- Cloud synchronisation across devices.
- Deadlines, reminders, priorities, and categories.
- Search, filtering, sorting, and recurring tasks.
- Database-based storage and network backup.

3.4 Feasibility Study

Technical Feasibility

The project is technically feasible because Python provides the language features and standard modules required. Tkinter is commonly distributed with Python, while JSON and OS file checks are provided by the standard library.

Economic Feasibility

The application can be developed and run with free software. It does not require paid hosting, a database server, or specialised hardware.

Operational Feasibility

The interface uses familiar controls such as a text field, list, and buttons. A user can learn the main operations with little instruction.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Hardware Requirements

- A desktop or laptop computer.
- A keyboard and pointing device.
- Basic memory and storage sufficient to run Python and save a small text file.
- A display capable of showing a standard desktop window.

4.2 Software Requirements

- Python 3.x.
- Tkinter support in the Python installation.
- An editor or IDE such as IDLE or Visual Studio Code.
- Windows, Linux, or macOS with a supported graphical desktop environment.

4.3 Modules and Responsibilities

Module / Component	Responsibility
tkinter	Creates the window and GUI controls such as labels, entry fields, list boxes, frames, and buttons.
messagebox	Displays warnings for empty input or missing task selection.
json	Reads and writes the task list using JSON format.
os	Checks whether the JSON storage file exists.
tasks list	Keeps task dictionaries in memory while the program runs.
tasks.json	Stores task records between application sessions.

4.4 Installation and Execution

11. Install a compatible Python 3 version.
12. Save the source code as todo_app.py.
13. Confirm Tkinter is available in the Python installation.
14. Run the file using an IDE or the command python todo_app.py.
15. Use the window to add, complete, and delete tasks.

5. TECHNOLOGIES USED

5.1 Python

Python is a high-level programming language known for readable syntax and a broad standard library. In this project, Python handles application logic, data structures, event callbacks, validation, and file operations. Functions separate the different task operations.

5.2 Tkinter GUI Library

Tkinter is Python's interface to the Tcl/Tk GUI toolkit. It provides widgets that receive user input and display information. The application uses Tk() for the main window, Label for headings and status text, Entry for task input, Listbox for the task list, Button for actions, and Frame to group controls.

5.3 JSON Data Format

JSON stands for JavaScript Object Notation. It is a text-based format for representing structured data. Python's json.dump() writes the list of task dictionaries to a file, and json.load() converts the stored JSON data back into Python objects.

```
[
  {"text": "Finish assignment", "done": false},
  {"text": "Revise notes", "done": true}
]
```

5.4 OS Module

The os module provides operating-system-related functionality. The application uses os.path.exists(FILE_NAME) to check whether tasks.json is present before trying to read it. This allows a first-time run to begin without a saved file.

5.5 Message Boxes

The messagebox module displays warning dialogs when the user attempts to add an empty task or performs an action that requires selecting a task but has not selected one.

5.6 Event-Driven Programming

The application responds to events such as button clicks and pressing Enter. A button's command property points to a function, while bind() connects the Return key to add_task(). root.mainloop() runs the event loop so the application can respond to user actions.

6. SYSTEM ANALYSIS AND DESIGN

6.1 System Architecture

The program has three cooperating parts: presentation, application logic, and local storage. The presentation layer contains Tkinter widgets. The application logic validates user actions and modifies the tasks list. The storage layer reads and writes tasks.json.

Layer	Main Elements	Purpose
Presentation	Entry, Listbox, Labels, Buttons	Collect input and display tasks and status.
Application logic	add_task, toggle_task, delete_task, clear_completed, refresh_tasks	Validate actions, modify data, and update the screen.
Persistence	load_tasks, save_tasks, JSON file	Load existing tasks and save changes.

6.2 Data Model

Each task is represented by a dictionary with two fields. The text field stores a string describing the task. The done field stores a Boolean value: False means pending, while True means completed.

```
task = {"text": "Prepare project report", "done": False}
```

6.3 Input, Processing and Output

Stage	Description
Input	Task text entered by the user or a selected task index.
Processing	Validate input, append or modify a dictionary, or filter the task list.
Storage	Write the current list to tasks.json.
Output	Update the Listbox, show warnings, and refresh the completion count.

6.4 User Interface Design

The window is titled “My To-Do List” and starts at 500 by 550 pixels with a minimum size of 400 by 450 pixels. A heading and subtitle appear at the top. The entry field and Add button are placed together, followed by the task list, completion counter, and three action buttons. Colours and spacing distinguish the controls.

7. SYSTEM WORKFLOW AND ALGORITHMS

7.1 Startup Workflow

16. Create the Tkinter root window.
17. Create and arrange labels, entry field, list box, frames, and buttons.
18. Call load_tasks() to read saved task data if the file exists.
19. Call refresh_tasks() to populate the task list and completion counter.
20. Start root.mainloop() to respond to user events.

7.2 Add Task Algorithm

21. Read text from the entry field and remove surrounding whitespace.
22. If the text is empty, show a warning and stop.
23. Append a dictionary containing the text and done=False to tasks.
24. Clear the entry field.
25. Save the updated list and refresh the display.

7.3 Toggle Task Algorithm

26. Read the selected row from the Listbox.
27. If no row is selected, display a warning and stop.
28. Use the selected row index to locate the task dictionary.
29. Reverse its done value using Boolean negation.
30. Save the updated data and refresh the display.

7.4 Delete and Clear Algorithms

For deletion, the program verifies that a row is selected, removes the task at that index, saves the updated list, and refreshes the display. For clearing completed tasks, it filters the list to retain only tasks whose done value is False.

```
tasks = [task for task in tasks if not task["done"]]
```

7.5 Workflow Diagram (Text Representation)

START → LOAD SAVED TASKS → DISPLAY TASKS → WAIT FOR USER ACTION

↓

VALIDATE INPUT / SELECTION → PERFORM OPERATION → SAVE JSON

↓

REFRESH TASK LIST AND COUNTER → WAIT FOR NEXT ACTION

8. IMPLEMENTATION AND CODE IMPLEMENTATION

8.1 Imports and Global Variables

```
import tkinter as tk
from tkinter import messagebox
import json
import os

FILE_NAME = "tasks.json"
tasks = []
```

The imports make GUI widgets, warning dialogs, JSON handling, and file-existence checks available. FILE_NAME stores the file name in one place, and tasks holds the current task records in memory.

8.2 Loading Saved Data

```
def load_tasks():
    global tasks
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as file:
                tasks = json.load(file)
        except (json.JSONDecodeError, OSError):
            tasks = []
```

The global statement allows the function to assign to the module-level tasks variable. If the file exists, it is opened in read mode and decoded. JSON Decode Error handles malformed JSON, while OSError handles common file errors. In these cases, tasks becomes an empty list.

8.3 Saving Data

```
def save_tasks():
    with open(FILE_NAME, "w") as file:
        json.dump(tasks, file, indent=4)
```

The file is opened in write mode, replacing its previous contents with the current task list. json.dump() serializes Python objects into JSON. The indent value makes the saved file easier to read.

8.4 Refreshing the GUI

```
task_list.delete(0, tk.END)
for task in tasks:
    status = "✓" if task["done"] else "○"
    task_list.insert(tk.END, f'{status}  {task["text"]}')
count = sum(task["done"] for task in tasks)
status_label.config(text=f"{count} of {len(tasks)} tasks completed")
```

The old Listbox items are deleted first to avoid duplicates. Each task is inserted with a symbol based on its done status. The sum expression counts completed tasks, and the label displays that count against the total.

8.5 Adding a Task

```
text = task_entry.get().strip()
if not text:
    messagebox.showwarning("Empty Task", "Please enter a task.")
    return
tasks.append({"text": text, "done": False})
task_entry.delete(0, tk.END)
save_tasks()
refresh_tasks()
```

The strip() method removes surrounding whitespace. A string containing only spaces is treated as empty. A valid task is stored with done=False, meaning it starts as pending. After saving, refresh_tasks() synchronizes the interface with the data.

8.6 Toggling Completion Status

```
selection = task_list.curselection()
if not selection:
    messagebox.showwarning("No Selection", "Select a task first.")
    return
index = selection[0]
tasks[index]["done"] = not tasks[index]["done"]
save_tasks()
refresh_tasks()
```

curselection() returns the selected Listbox position. The program checks for an empty selection before accessing the list. Boolean negation changes True to False or False to True, allowing the same button to mark a task done or undo completion.

8.7 Deleting and Clearing Completed Tasks

```
index = selection[0]
del tasks[index]

# Keep only pending tasks
tasks = [task for task in tasks if not task["done"]]
```

Deleting a selected task removes the dictionary at the selected index. Clearing completed tasks filters the whole list and retains only pending tasks. After either operation, the file and interface are updated.

8.8 GUI Setup and Main Loop

The main window and widgets are created after the function definitions. The pack() geometry manager arranges widgets, while Frame groups related controls. The Entry

widget is bound to the Return key using a lambda callback. Each Button has a command connected to its operation. Finally, the program calls load_tasks(), refresh_tasks(), and root.mainloop().

8.9 Error Handling

The application checks for empty task descriptions and missing selections before changing the task list. Loading is protected against invalid JSON and operating-system read errors. The current save_tasks() function does not explicitly handle write failures, which is a possible improvement.

9. TESTING AND RESULTS

Testing checks whether the application responds correctly to common user actions and boundary cases. The following table lists expected outcomes that can be checked by running the program. These are test plans and expected results, not a claim that an automated test suite was executed.

ID	Test / Input	Expected Result
TC01	Enter a valid task and click Add	Pass
TC02	Click Add with an empty field	Pass
TC03	Enter spaces only and click Add	Pass
TC04	Select a pending task and click Mark Done / Undo	Pass
TC05	Select a completed task and click Mark Done / Undo	Pass
TC06	Click Delete without selecting a task	Pass
TC07	Select a task and click Delete	Pass
TC08	Click Clear Completed	Pass
TC09	Close and reopen after saving	Pass
TC10	Run when tasks.json is missing	Pass
TC11	Press Enter in the entry field	Pass
TC12	Delete the last task	Pass

9.1 Expected Output

When launched, the application displays a window titled “My To-Do List.” A task is shown with an open-circle symbol when pending and a check mark when completed. The status label indicates completed tasks out of the total. Warning dialogs appear for empty input or missing selections.

9.2 Result Discussion

The design keeps the visible interface and stored task list synchronized by calling save_tasks() and refresh_tasks() after operations that change data.

10. LIMITATIONS, FUTURE ENHANCEMENTS, CONCLUSION AND REFERENCES

10.1 Limitations

- Data is saved locally and is not synchronised across devices.
- There is no login system or separation between users.
- Tasks do not currently support due dates, priorities, categories, or reminders.
- The interface does not provide search, filters, or sorting.
- Save errors are not explicitly caught, and malformed JSON causes the app to start with an empty task list.
- Tasks are stored in the program's working directory, which users should know when backing up data.

10.2 Future Enhancements

- Add due dates, priorities, labels, and categories.
- Provide task search and filters for pending and completed items.
- Add reminder notifications and recurring tasks.
- Add a confirmation prompt before deleting tasks.
- Handle write errors and provide safer backup and recovery.
- Use SQLite for structured storage or add optional cloud synchronisation.
- Improve accessibility, keyboard navigation, and visual themes.

10.3 Conclusion

The Python To-Do List Application is a practical example of a small event-driven desktop program. It uses Tkinter to provide a graphical interface, Python data structures to represent tasks, and JSON file handling to retain information. The main operations—adding tasks, toggling completion, deleting a selected task, and clearing completed tasks—are separated into functions for clarity. The project reinforces functions, lists, dictionaries, conditional logic, loops, validation, exception handling, file input/output, and GUI events.

10.4 References

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Python Software Foundation. json — JSON encoder and decoder. https://docs.python.org/3/library/json.html
- Python Software Foundation. os — Miscellaneous operating system interfaces. https://docs.python.org/3/library/os.html